Recitation 1 – Intro to Computer Networks

Daniel Karalnik and Liam Tal

Computer Networks

Table of Contents

1. Administration
2. Circuit and Packet Switching
3. Application Layer Protocols
4. Network Layer Model
5. Internet Structure

Homework and Grade Structure

- Exercises **individually** (not in pairs)
  - 5 "dry" exercises (**15%**)
  - Appeals: up to **three days** after grading
- Final exam (**85%**) - **Must Pass**

Office Hours & Contact Information

Instructor: Daniel Karalnik

- Email: `daniel.karalnik@post.runi.ac.il`
- Office hours: **Sunday 9:00-10:00** (by appointment)
- Location: **Zoom** - `https://runi-ac-il.zoom.us/j/82921232959`
- **Must email by Saturday evening** with the subject you wish to discuss

Grader: Liam Tal

- Email: `liam.tal@post.runi.ac.il`

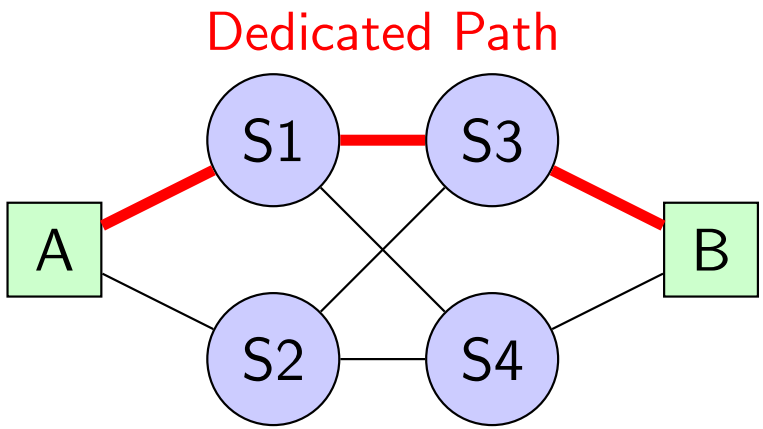
We encourage you to ask questions on the Piazza Forum!

Today

- Circuit vs. Packet Switching (recap)
- **Calculation:** transmission speeds in packet switching & **performance comparison**
- **What is a protocol?** (application layer) + **Sequence diagrams**
- **Layer activity** in different communications
- **Internet structure & players**

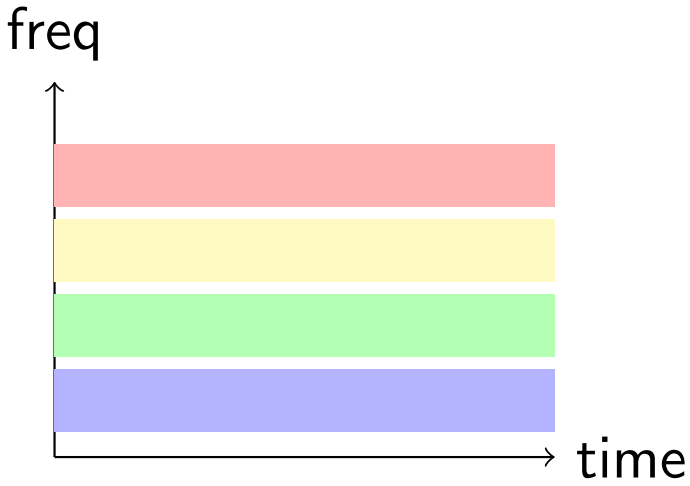
Circuit-Switching (recap)

- Call (connection) **setup**; reserved resources per call.
- Predictable bandwidth and delay while call is active.
- Inefficient when traffic is bursty or idle.



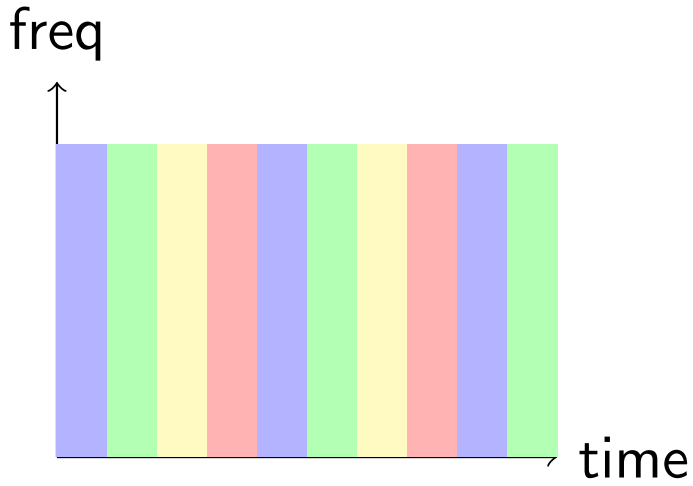
Multiplexing: FDM and TDM

FDM (Frequency Division)



Constant frequency bands

TDM (Time Division)

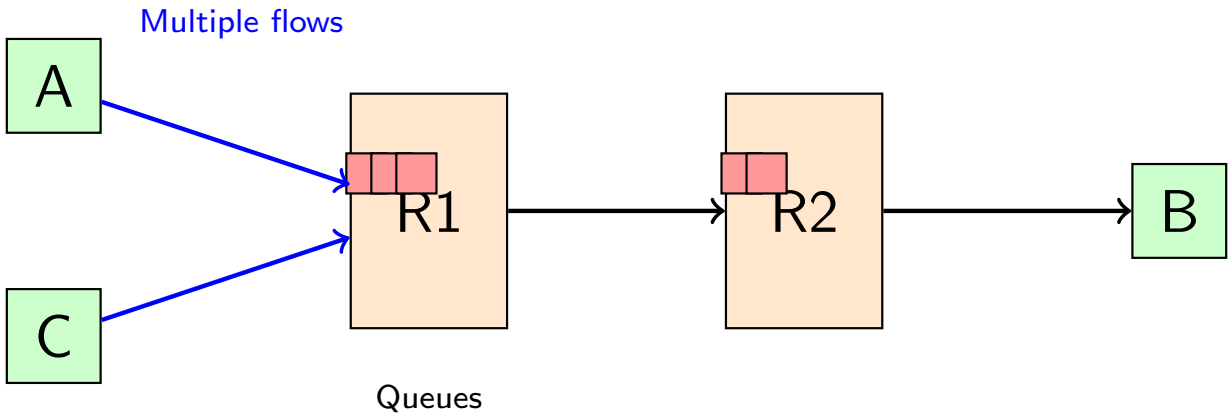


Rotating time slots

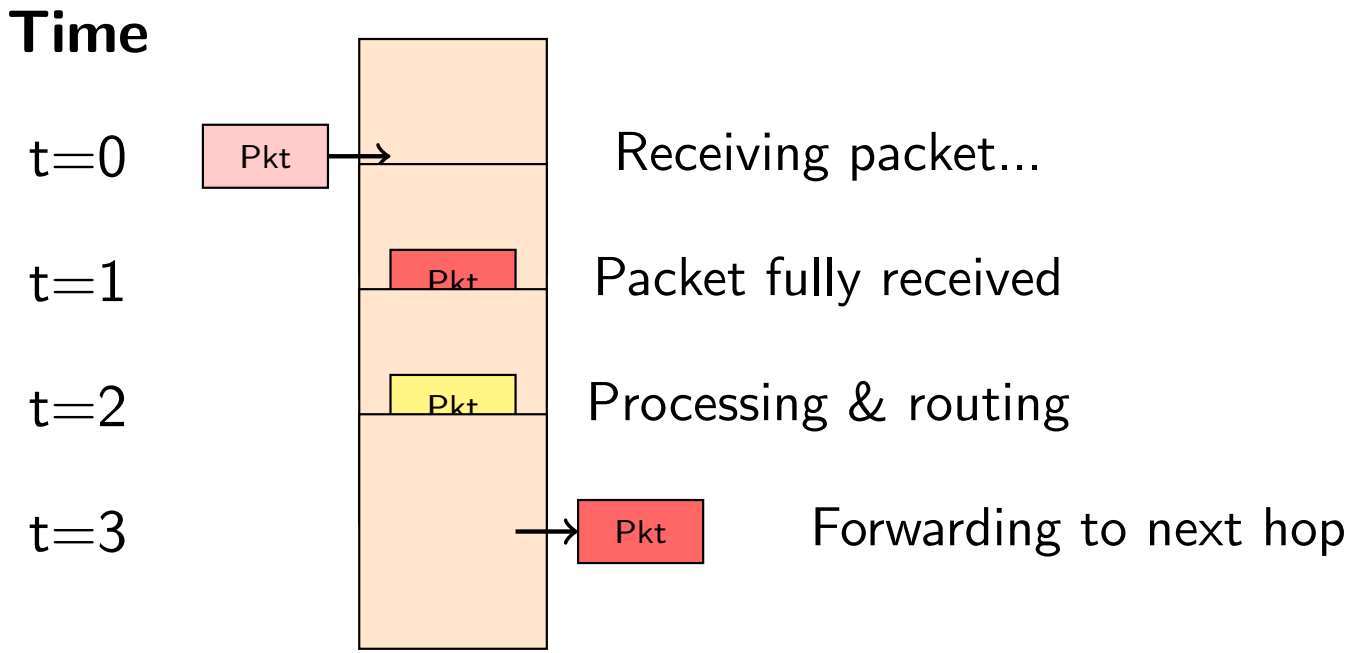
- **FDM:** Each user gets a dedicated frequency band
- **TDM:** Each user gets periodic time slots
- Both achieve same per-user bandwidth in circuit switching

Packet Switching (recap)

- No call setup; routers *store-and-forward* packets.
- Efficient statistical multiplexing; shares link among active flows.
- Variable delay & potential loss under load.



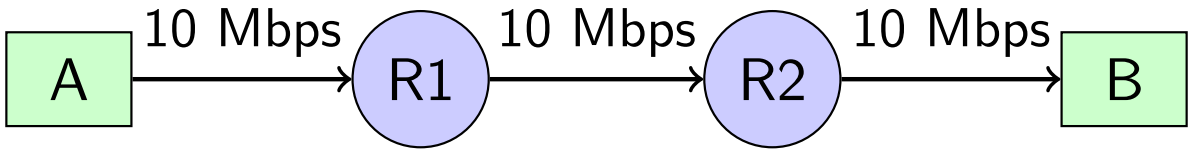
Store-and-Forward: How Packet Switching Works



Router must receive and process **entire packet** before forwarding → introduces delay at each hop (more on this next week).

Performance Question: Throughput & Latency

Scenario. Two hosts A→B over 3 links (each $R = 10$ Mbps).



- **Circuit switching TDM:** 10 equal time slots; call setup = 300 ms.
- **Circuit switching FDM:** 10 equal frequency channels; call setup = 300 ms.
- **Packet switching:** packets of 1,500 B; per-hop processing negligible.
- File size: $F = 12$ MB.

Tasks. (a) Time under TDM. (b) Time under FDM. (c) Time under packet switching. (d) Discuss.

Solution Sketch - Calculations

(a) TDM Circuit switching. Per-user rate $= 10/10 = 1$ Mbps.
Transmit $F = 12 \cdot 8 = 96$ Mbit $\Rightarrow$ 96 s, plus setup 0.3 s $\Rightarrow$ $\boxed{96.3 \text{ s}}$.

(b) FDM Circuit switching. Per-user bandwidth $= 10/10 = 1$ Mbps.
Same calculation: 96 s transmission $+0.3$ s setup $\Rightarrow$ $\boxed{96.3 \text{ s}}$.

Why TDM = FDM? Different division strategies, same result:
- **TDM:** Full BW (10 Mbps) $\times$ fraction of time $(\frac{1}{10}) = 1$ Mbps
- **FDM:** Fraction of BW $(\frac{10}{10} = 1$ Mbps) $\times$ all the time $(1) = 1$ Mbps
- Both partition the *bandwidth*$\times$*time product* equally!

(c) Packet switching (idle path). Bottleneck rate $= 10$ Mbps.
No setup delay! Time $= 96/10 = \boxed{9.6 \text{ s}}$.

Solution Sketch - Discussion

(d) Comparison Table:

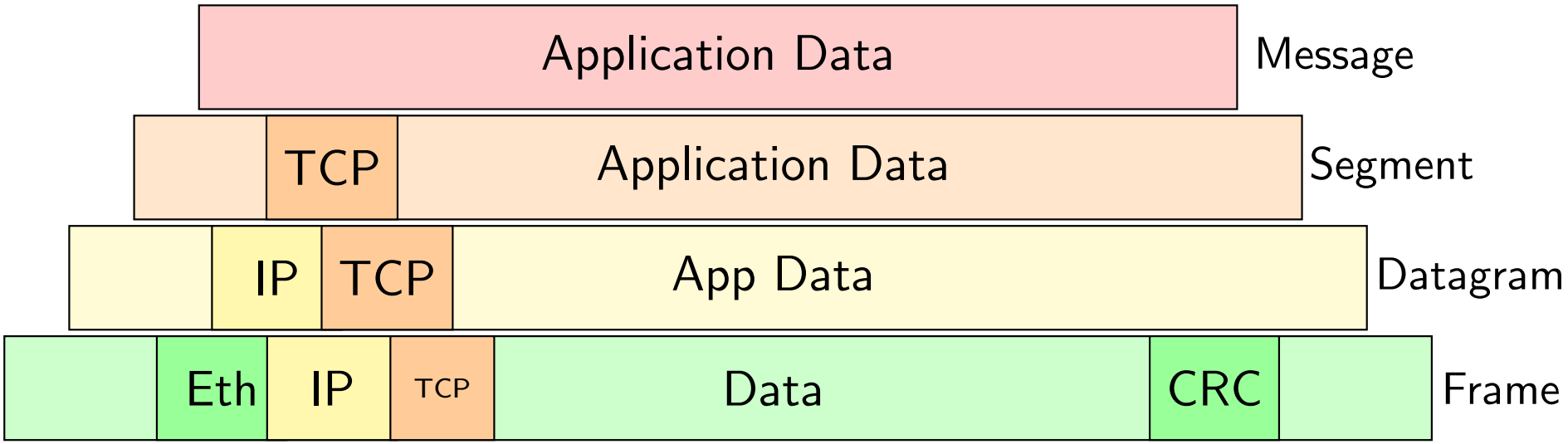
Method	**Time**	**Advantages**	**Disadvantages**
TDM/FDM	96.3s	Predictable, guaranteed	Slow, setup, poor util
Packet	9.6s	Fast, no setup, efficient	Variable delay under load

Key insight: TDM vs FDM doesn't matter – both reserve $\frac{1}{10}$ bandwidth. Packet wins here!

What is a Protocol?

- A set of **rules** governing message formats and the order of messages exchanged by entities.
- **Critical Distinction:**
  - **Valid communication:** messages well-formed, in correct order (syntax/format correct)
  - **Correct task execution:** the application achieves its goal (semantics/business logic correct)
- Example: A protocol exchange can be *technically valid* but still fail at the application level (e.g., authentication fails, password too weak, insufficient permissions).

Protocol Encapsulation: How Layers Work Together



Each layer adds its own header to the data from the layer above.

Design a Simple Application-Layer Protocol

Task: Change a password between Client and Server.

Define messages and flow so that both sides can interoperate.

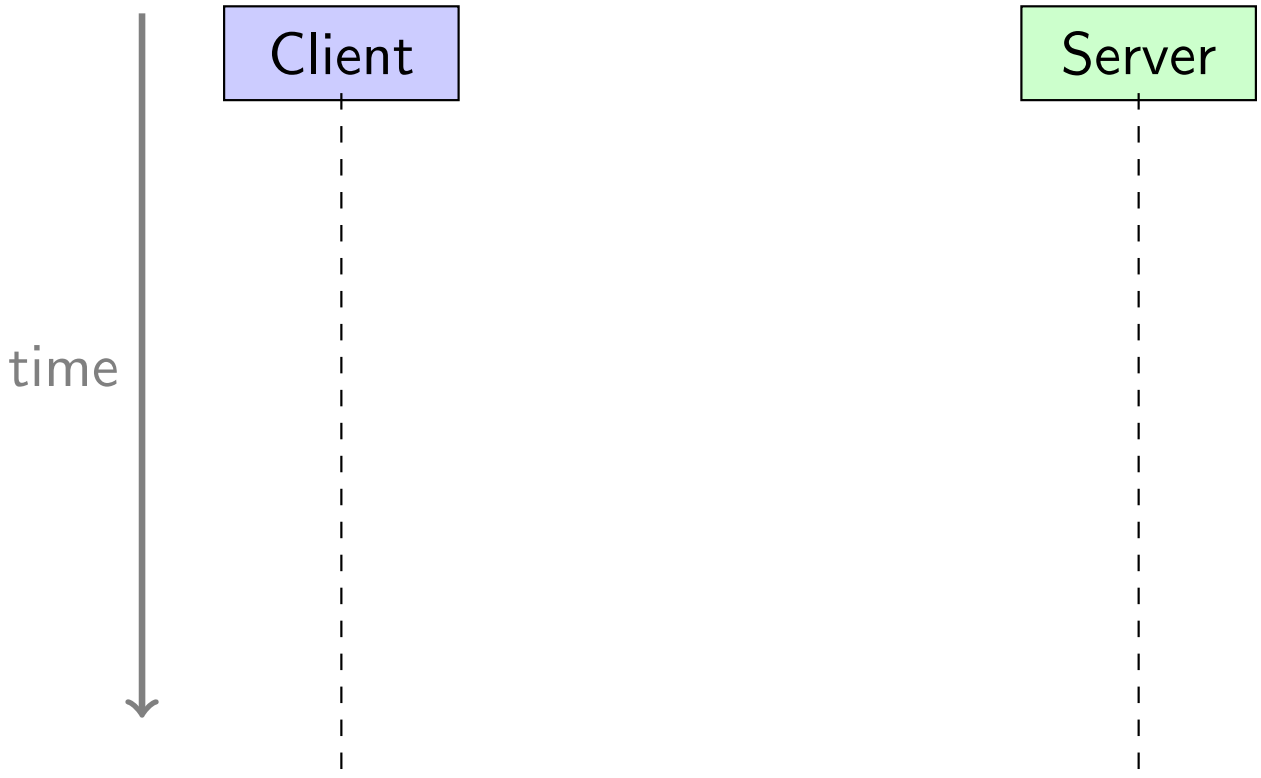
- `HELLO` → `HELLO_ACK`
- `AUTH_REQ{user}` → `CHALLENGE{nonce}`
- `AUTH_RESP{HMAC(nonce, old_pw)}` → `AUTH_OK/FAIL`
- `CHPASS{new_pw}` → `CHPASS_OK / CHPASS_ERR{reason}`

Point: A run may be *valid* (all messages well-formed) while the task still fails, e.g., `CHPASS_ERR{weak_pw}`.

Note: This application-layer protocol relies on lower layers (Transport/Network) for delivery.

Sequence Diagram (Concept)

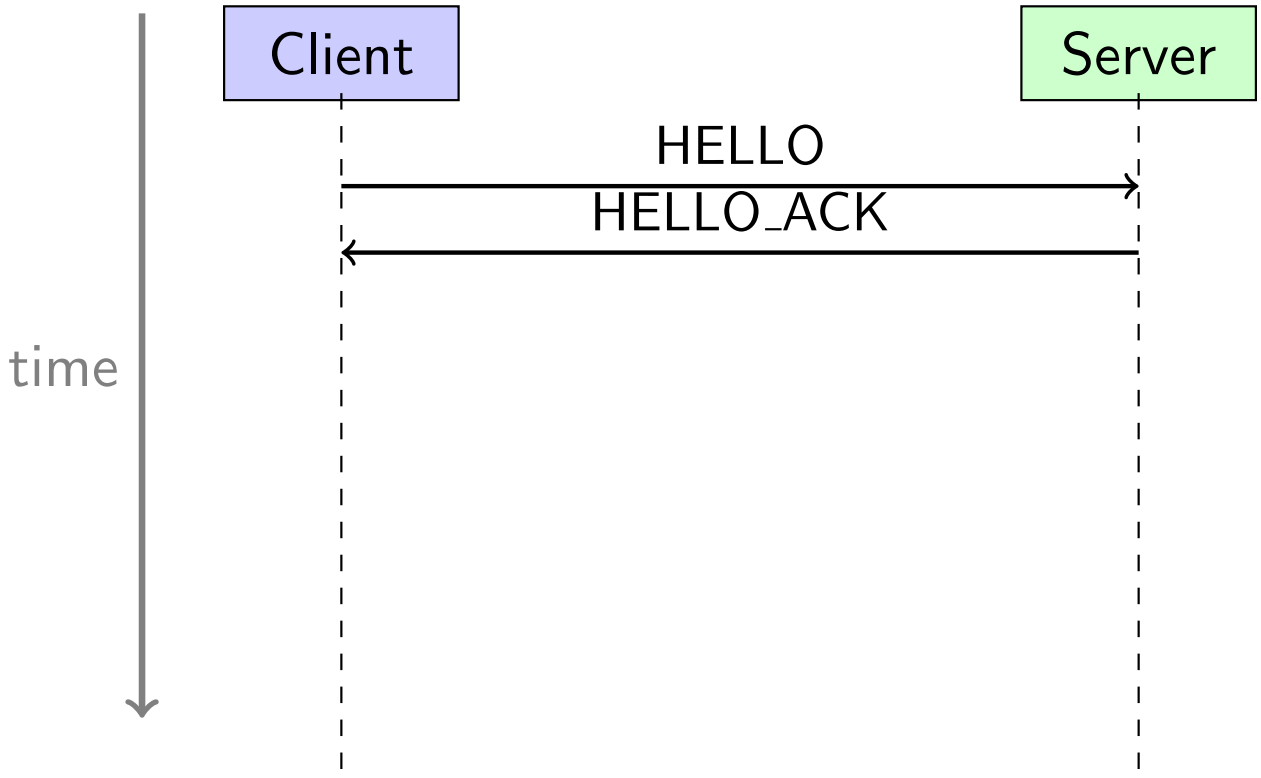
A *sequence diagram* visualizes **who talks to whom** and **in what order**.



Industry Practice: Developers may use sequence diagrams (or UMLs) to design **application protocols**. Essential for documenting APIs, microservices, distributed systems etc.

Sequence Diagram (Concept)

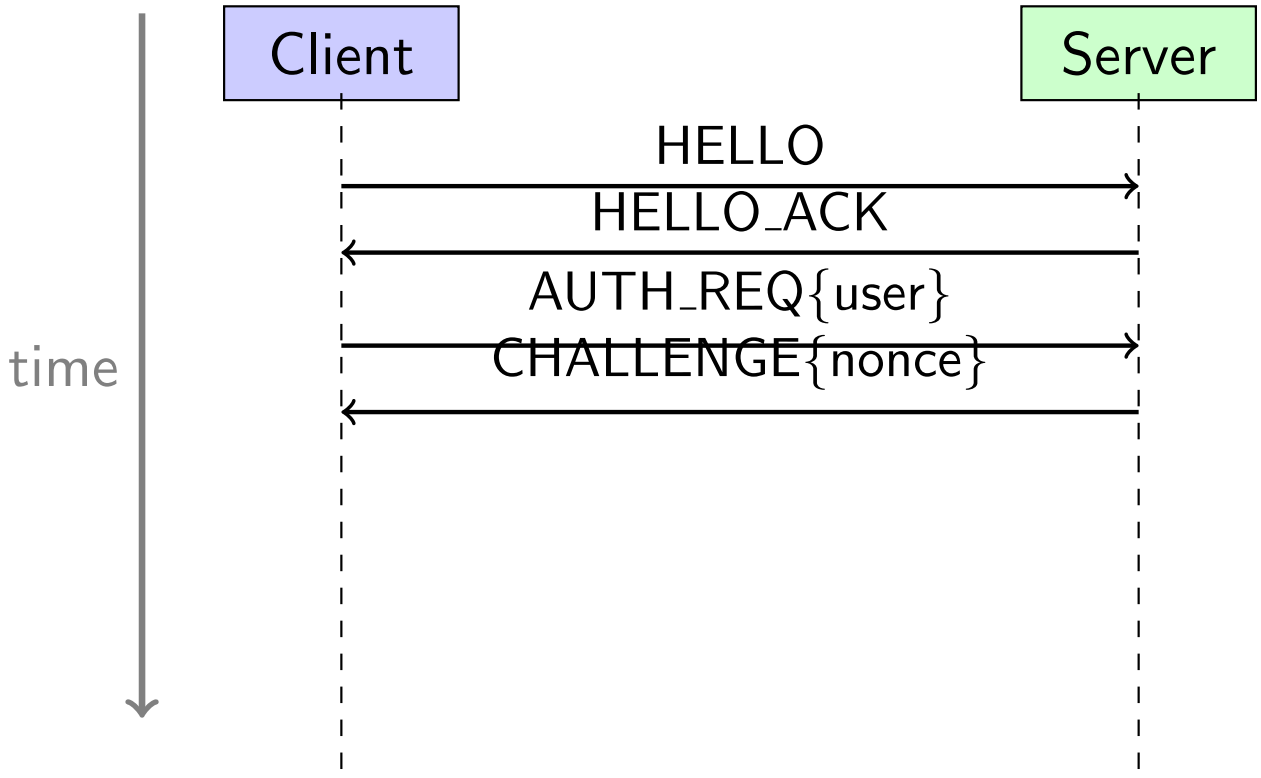
A *sequence diagram* visualizes **who talks to whom** and **in what order**.



Industry Practice: Developers may use sequence diagrams (or UMLs) to design **application protocols**. Essential for documenting APIs, microservices, distributed systems etc.

Sequence Diagram (Concept)

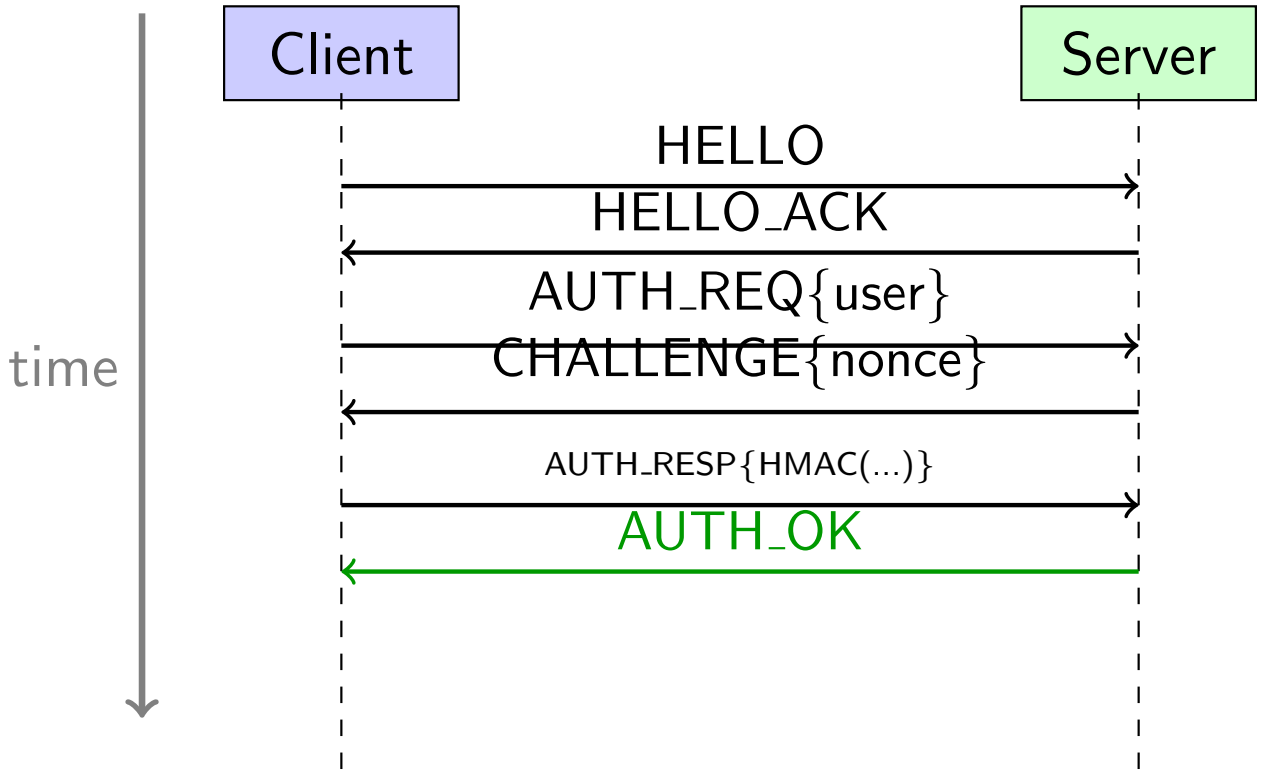
A *sequence diagram* visualizes **who talks to whom** and **in what order**.



Industry Practice: Developers may use sequence diagrams (or UMLs) to design **application protocols**. Essential for documenting APIs, microservices, distributed systems etc.

Sequence Diagram (Concept)

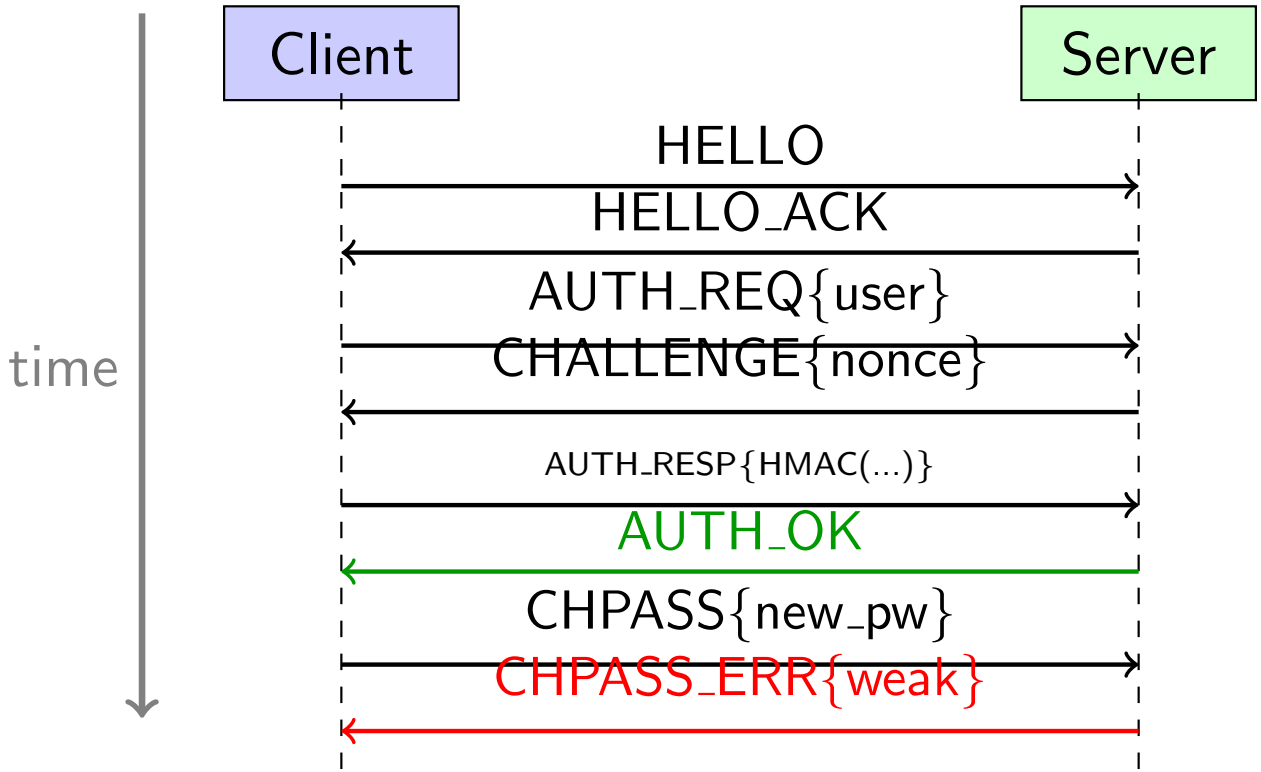
A *sequence diagram* visualizes **who talks to whom** and **in what order**.



Industry Practice: Developers may use sequence diagrams (or UMLs) to design **application protocols**. Essential for documenting APIs, microservices, distributed systems etc.

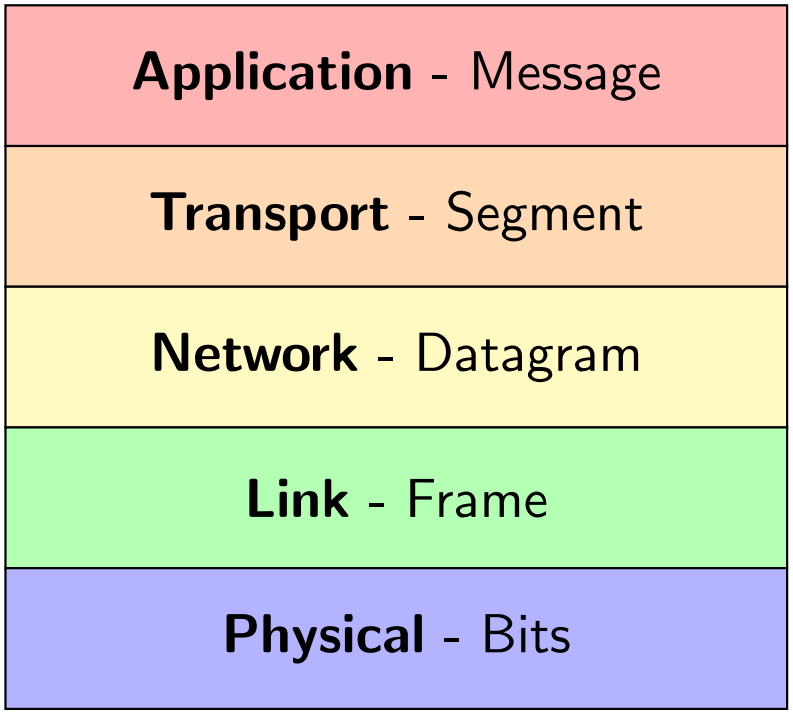
Sequence Diagram (Concept)

A *sequence diagram* visualizes **who talks to whom** and **in what order**.



Industry Practice: Developers may use sequence diagrams (or UMLs) to design **application protocols**. Essential for documenting APIs, microservices, distributed systems etc.

The 5-Layer Network Model



Protocols:

- App: HTTP/S
- Trans: TCP, UDP
- Net: IP (IPV4, IPV6)
- Link: Ethernet (protocol)
- Phys: IEEE 802.11 specification (internet cables), IEEE 802.3 specification (WiFi)

Each layer provides services to the layer above and uses services from the layer below.

Layer Identification Exercise (Q1)

Goal: Identify which layers are active and understand their general purpose.

Scenario: You open your web browser and visit `www.google.com`. The search page appears on your screen.

Question: Which of the 5 layers are active? What is the *general purpose* of each layer in this scenario?

Layer Identification Exercise (Q1)

Goal: Identify which layers are active and understand their general purpose.

Scenario: You open your web browser and visit `www.google.com`. The search page appears on your screen.

Question: Which of the 5 layers are active? What is the *general purpose* of each layer in this scenario?

Think about:

- What is your browser doing?
- How does data travel from Google to you?
- What physical connection are you using?

Note: Don't worry about technical protocol names yet – just think about the purpose!

Solution (Q1)

All 5 layers are active!

1. **Application Layer:**
   Your browser communicates with Google's web server to request and receive the webpage content.
2. **Transport Layer:**
   Ensures all the webpage data arrives reliably and in the correct order.
3. **Network Layer:**
   Finds the route from your computer through the Internet to Google's servers.
4. **Link Layer:**
   Transmits data over your local network (WiFi or Ethernet cable to your router).
5. **Physical Layer:**
   The actual electrical signals on cables or radio waves through the air.

Coming soon: In the next weeks, we'll learn *how* each layer does its job (protocols, headers, algorithms, etc.)

Layer Identification Exercise (Q2)

Goal: Compare different applications and how they use the same 5-layer structure.

Scenario: You're making a video call with a friend on Zoom (or Skype, FaceTime, etc.).

Question: Do video calls use the same 5 layers? What might be *different* about video calls compared to web browsing?

Layer Identification Exercise (Q2)

Goal: Compare different applications and how they use the same 5-layer structure.

Scenario: You're making a video call with a friend on Zoom (or Skype, FaceTime, etc.).

Question: Do video calls use the same 5 layers? What might be *different* about video calls compared to web browsing?

Think about:

- Is real-time video the same as loading a webpage?
- What happens if some video data is lost or delayed?
- Do you need *all* the data, or just *enough* data?

Hint: Different applications have different priorities!

Solution (Q2)

Yes! All 5 layers are still active.

1. **Application Layer:**
   Zoom app handles video/audio encoding, compression, and displaying your friend's video.
2. **Transport Layer:**
   Sends video/audio data *quickly* – different strategy than web browsing! (Speed $>$ perfect reliability)
3. **Network Layer:**
   Routes your video stream through the Internet to your friend's computer.
4. **Link Layer:**
   Transmits over your home network (same as web browsing).
5. **Physical Layer:**
   WiFi or cable connection (same as web browsing).

Key difference: Video calls prioritize *speed* over *perfect reliability*. A few lost frames are OK, but delays are not! We'll learn why in the Transport

How is the Internet Organized?

Key Question: The Internet connects billions of devices worldwide. How is this massive network structured?

The Internet is NOT:

- A single giant network owned by one company
- A random mesh of connections

The Internet IS:

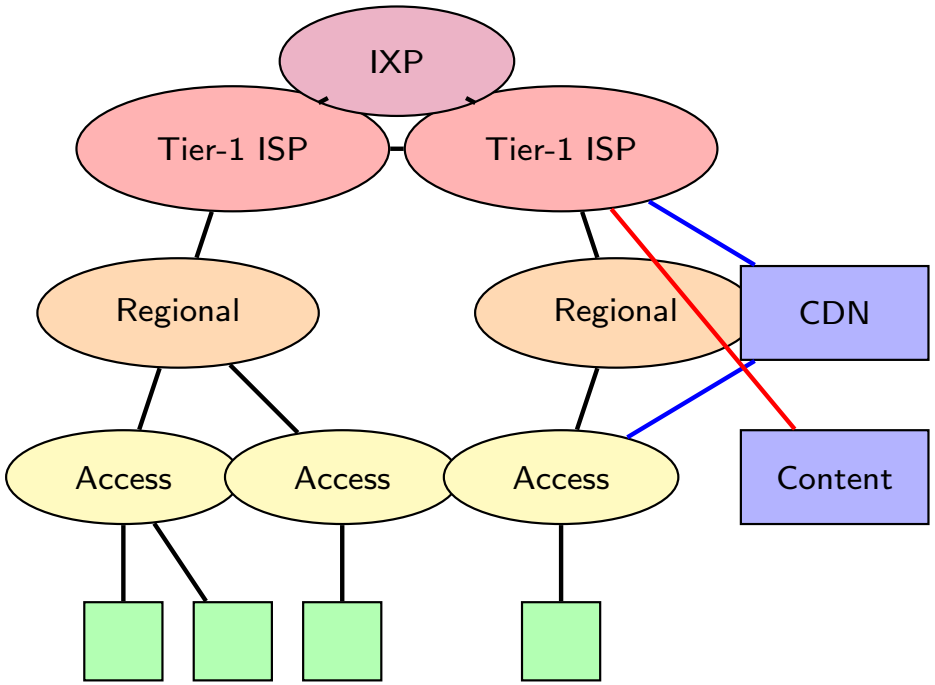
- A **network of networks**
- A **hierarchical structure** with different types of organizations
- A combination of **cooperation and competition**

How is the Internet Organized?

Key Players:

- **ISPs** (Internet Service Providers) - connect networks together
- **IXPs** (Internet Exchange Points) - where ISPs meet and exchange traffic
- **CDNs** (Content Delivery Networks) - cache popular content close to users
- **Content Providers** - companies like Google, Netflix that provide content

Internet Hierarchy: The ISP Pyramid



1. **IXP**
   - Physical location where Tier 1 ISPs connect.
2. **Tier-1 ISPs** (Top)
   - Global backbone
   - Connect to *all* other Tier-1s
3. **Regional ISPs**
   - Cover countries/regions
4. **Access ISPs**
   - Last mile to homes
5. **End Users**
   - You and me!

Special Players: IXPs, CDNs, and Content Providers

IXPs (Internet Exchange Points):

- Physical locations where ISPs connect and exchange traffic
- **Why?** Cheaper and faster than going through Tier-1s
- Examples: AMS-IX (Amsterdam), DE-CIX (Frankfurt), LINX (London)
- Business model: ISPs pay for ports, get direct peering

CDNs (Content Delivery Networks):

- Cache popular content *close to users*
- **Why?** Faster delivery, less traffic through backbone
- Examples: Cloudflare, Akamai, Fastly
- Connect at multiple levels (Tier-1, Access ISPs)

Special Players: IXPs, CDNs, and Content Providers

Content Providers:

- Large companies producing content (Google, Netflix, AWS)
- Often build their *own private networks*
- Connect directly to Tier-1s (bypass the hierarchy!)
- Netflix traffic = ~15% of all Internet traffic!

Example: Watching a YouTube Video

Scenario: You're in Tel Aviv, watching a video on YouTube.

Path 1: Traditional (Rare Today)

1. Your device → WiFi router
2. Router → **Bezeq** (Access ISP)
3. Bezeq → **Tier-1 ISP**
4. Tier-1 ISP → Another Tier-1 or Google's network
5. Google servers → Video data back

Path 2: With CDN (Common Today)

1. Your device → WiFi router
2. Router → **Bezeq** (Access ISP)
3. Bezeq → **Google CDN server in Israel!**
4. Video served from local cache (fast!)

Why CDNs matter: Path 1 might take 100-200ms. Path 2 takes 10-20ms. Video starts playing **10x faster**! Plus saves bandwidth for ISPs.